

CSS Theory Questions

1. Overview of CSS and Its Importance in Web Design

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation of HTML documents. It allows developers to separate content (HTML) from design (CSS), making web development more efficient and organized.

Importance of CSS in Web Design:

- **Separation of Concerns:** Keeps structure (HTML) separate from design (CSS).
 - **Consistency:** Ensures uniform design across multiple web pages.
 - **Reusability:** External stylesheets can be used across different pages.
 - **Faster Page Load:** CSS files are cached, reducing load time.
 - **Responsive Design:** Allows styling for different screen sizes and devices.
-

2. Types of CSS

2.1 Inline CSS

- Written directly within an HTML element's style attribute.
- Example:
- `<h1 style="color: blue;">Hello World</h1>`
- **Use Case:** Quick, one-off styling of specific elements.

2.2 Internal CSS

- Defined within a `<style>` tag inside the `<head>` section of the HTML document.
- Example:
- `<style>`
- `p { color: green; }`
- `</style>`
- **Use Case:** Styling a single HTML document without affecting others.

2.3 External CSS

- Written in a separate .css file and linked using the <link> tag.
 - Example:
 - <link rel="stylesheet" href="styles.css">
 - **Use Case:** Ideal for maintaining consistent design across multiple pages.
-

3. Definition and Difference Between Margin and Padding

3.1 Margin

- Margin is the space **outside** an element's border.
- It creates space between the element and other surrounding elements.

3.2 Padding

- Padding is the space **inside** an element's border.
- It creates space between the content and the border of the element.

3.3 Key Differences

Feature	Margin	Padding
Location	Outside the border	Inside the border
Affects	Distance from other elements	Content spacing within the element
Background Color	Does not extend into margin	Background fills padding area

4. Introduction to CSS Pseudo-Classes

4.1 What are Pseudo-Classes?

Pseudo-classes are used to define the special state of an element. They allow you to apply styles based on user interaction or element conditions.

4.2 Common Pseudo-Classes:

Pseudo-Class Description

:hover	Applies when the user hovers over an element
:focus	Applies when an element (like an input) is focused
:active	Applies when an element is being clicked
:visited	Applies to visited links
:first-child	Applies to the first child of a parent element

4.3 Use Cases

- Creating interactive navigation menus.
 - Highlighting input fields on focus.
 - Styling elements differently during user interaction.
-

5. Difference Between ID and Class in CSS

5.1 ID Selector

- Targets a single, **unique** element.
- Syntax: #idName
- Example:
- #header { color: red; }

5.2 Class Selector

- Targets **multiple elements** that share the same class name.
- Syntax: .className
- Example:
- .highlight { background-color: yellow; }

5.3 Key Differences

Feature	ID Selector	Class Selector
Uniqueness	Unique per page	Can be reused
Syntax	#id	.class
Specificity	Higher specificity	Lower specificity

5.4 Usage Scenarios

- Use **ID** when you want to apply a unique style to one element (e.g., header).
- Use **Class** when you want to apply the same style to multiple elements (e.g., buttons, boxes).

Module 16

6. CSS Selectors & Styling – Theory Assignment

6.1 What is a CSS selector? Provide examples of element, class, and ID selectors.

A **CSS selector** is a pattern used to select and apply styles to specific HTML elements. It tells the browser which HTML elements the CSS rules apply to.

- **Element Selector:** Selects all elements of a specific type.
 - `p {`
 - `color: blue;`
 - `}`
- **Class Selector:** Selects all elements with a specific class.
 - `.highlight {`
 - `background-color: yellow;`
 - `}`
- **ID Selector:** Selects a single element with a specific ID.
 - `#header {`
 - `font-size: 24px;`
 - `}`

6.2 Explain the concept of CSS specificity. How do conflicts between multiple styles get resolved?

CSS specificity is a hierarchy that determines which style rules are applied when multiple rules target the same element. The browser uses specificity to resolve conflicts.

- Inline styles have the highest specificity.
- ID selectors are more specific than class selectors.
- Class selectors are more specific than element selectors.

Example:

```
p { color: black; }      /* Element selector - low specificity */  
.text { color: green; }  /* Class selector - medium specificity */  
#mainText { color: red; } /* ID selector - high specificity */
```

In this case, the element with ID mainText will have red color due to higher specificity.

6.3 What is the difference between internal, external, and inline CSS? Discuss the advantages and disadvantages of each approach.

CSS Type	Description	Advantages	Disadvantages
Inline CSS	CSS written inside an element's style attribute	Quick for small changes	Hard to maintain; not reusable
Internal CSS	CSS written in a <style> tag inside <head>	Keeps styles in one HTML file	Not reusable across pages; harder with large sites
External CSS	CSS stored in a separate .css file	Reusable across multiple pages; easy to maintain	Requires additional HTTP request to load the file

7. CSS Box Model – Theory Assignment

7.1 Explain the CSS box model and its components (content, padding, border, margin). How does each affect the size of an element?

The **CSS box model** represents how elements are structured and spaced on a web page. It consists of:

1. **Content** – The actual content (text, image, etc.).
2. **Padding** – Space between content and border.
3. **Border** – The edge around padding.
4. **Margin** – Space outside the border, between elements.

Total width = content + padding + border + margin

Total height = content + padding + border + margin

Each part contributes to the element's total space on the page.

7.2 What is the difference between border-box and content-box box-sizing in CSS? Which is the default?

- **content-box** (default):
Width and height include only the content. Padding and border are added outside this box.
- **border-box**:
Width and height include the content, padding, and border. It makes it easier to size elements predictably.

Example:

box-sizing: border-box;

Default: content-box

8. CSS Flexbox – Theory Assignment

8.1 What is CSS Flexbox, and how is it useful for layout design? Explain the terms flex-container and flex-item.

CSS Flexbox is a layout model that allows easy alignment, spacing, and distribution of elements inside a container, especially when their size is dynamic or unknown.

- **Flex-container:** The parent element with display: flex.
- **Flex-item:** The child elements inside the flex container.

Flexbox simplifies creating responsive and flexible web layouts without using floats or positioning.

8.2 Describe the properties justify-content, align-items, and flex-direction used in Flexbox.

- **justify-content:** Aligns flex items horizontally along the main axis.
 - justify-content: center; /* other values: flex-start, flex-end, space-between */
 - **align-items:** Aligns flex items vertically along the cross axis.
 - align-items: center; /* other values: flex-start, flex-end, stretch */
 - **flex-direction:** Sets the direction of the main axis.
 - flex-direction: row; /* or column, row-reverse, column-reverse */
-

8. CSS Grid – Theory Assignment

8.1 Explain CSS Grid and how it differs from Flexbox. When would you use Grid over Flexbox?

CSS Grid is a two-dimensional layout system (rows and columns), while **Flexbox** is one-dimensional (either rows or columns, not both simultaneously).

- Use **Grid** when designing complex page layouts with rows and columns.
 - Use **Flexbox** when aligning items in a single row or column.
-

8.2 Describe the grid-template-columns, grid-template-rows, and grid-gap properties. Provide examples of how to use them.

- **grid-template-columns**: Defines column sizes.
- `grid-template-columns: repeat(3, 1fr);`
- **grid-template-rows**: Defines row sizes.
- `grid-template-rows: 100px 200px;`
- **grid-gap**: Sets spacing between rows and columns.
- `grid-gap: 20px;`

Example:

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-gap: 20px;  
}
```

9. Responsive Web Design with Media Queries – Theory Assignment

9.1 What are media queries in CSS, and why are they important for responsive design?

Media queries allow CSS to be applied based on the device's screen size, resolution, orientation, or other features.

They are essential for **responsive design**, which ensures websites look and function well on all devices (desktop, tablet, mobile).

9.2 Write a basic media query that adjusts the font size of a webpage for screens smaller than 600px.

```
@media (max-width: 600px) {  
  body {  
    font-size: 14px;  
  }  
}
```

This query reduces the font size on small screens to improve readability and layout.

10. Typography and Web Fonts – Theory Assignment

10.1 Explain the difference between web-safe fonts and custom web fonts. Why might you use a web-safe font over a custom font?

- **Web-safe fonts:** Fonts installed on most operating systems (e.g., Arial, Times New Roman). They load faster and don't rely on external sources.
- **Custom web fonts:** Fonts loaded from services like Google Fonts.

Why use web-safe fonts?

- Faster loading
- Guaranteed compatibility across devices
- No external dependencies

However, custom fonts allow for unique branding and style.

10.2 What is the font-family property in CSS? How do you apply a custom Google Font to a webpage?

- **font-family** defines the typeface used for text in an element.

Example:

```
body {  
  
  font-family: 'Roboto', sans-serif;  
  
}
```

To use a Google Font:

1. Add the <link> to HTML <head>:

```
<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"  
rel="stylesheet">
```

2. Use it in CSS:

```
body {  
  
  font-family: 'Roboto', sans-serif;  
  
}
```